STATIC & CONSTANT KEYWORDS IN C++

WHY STATIC & CONST MATTER

- Prevent accidental modification of data
- Improve program safety
- Help in real-world modeling
- Reduce memory usage (static members)
- Make code cleaner and more maintainable

The “this” Pointer

Definition:

- this is an implicit pointer available inside all non-static member functions
- It stores the address of the current calling object

Key Points:

- Used to differentiate data members from parameters
- Helps access object’s own variables
- Points to the current object

Common Uses:

- Assigning values when parameter name = data member name
- Printing object address
- Accessing object data explicitly

Example Concept:

- **this->x = x;**
- **(*this).x = x;**

CONSTANT DATA MEMBERS AND CONSTANT MEMBER FUNCTIONS

Definition: (Constant Data Member)

- Data members whose values cannot be changed after initialization

Rules:

- Must be initialized using initializer list
- Cannot be modified later
- Helps protect critical data
- (e.g., ID, roll number)

Definition:(Constant Member Function)

- Functions that do not modify object data

Syntax:

returnType functionName() const;

Key Points:

- Prevent accidental data changes
- Can be called by const objects
- Cannot modify data members

Definition:

- Used to initialize class data members before constructor body executes

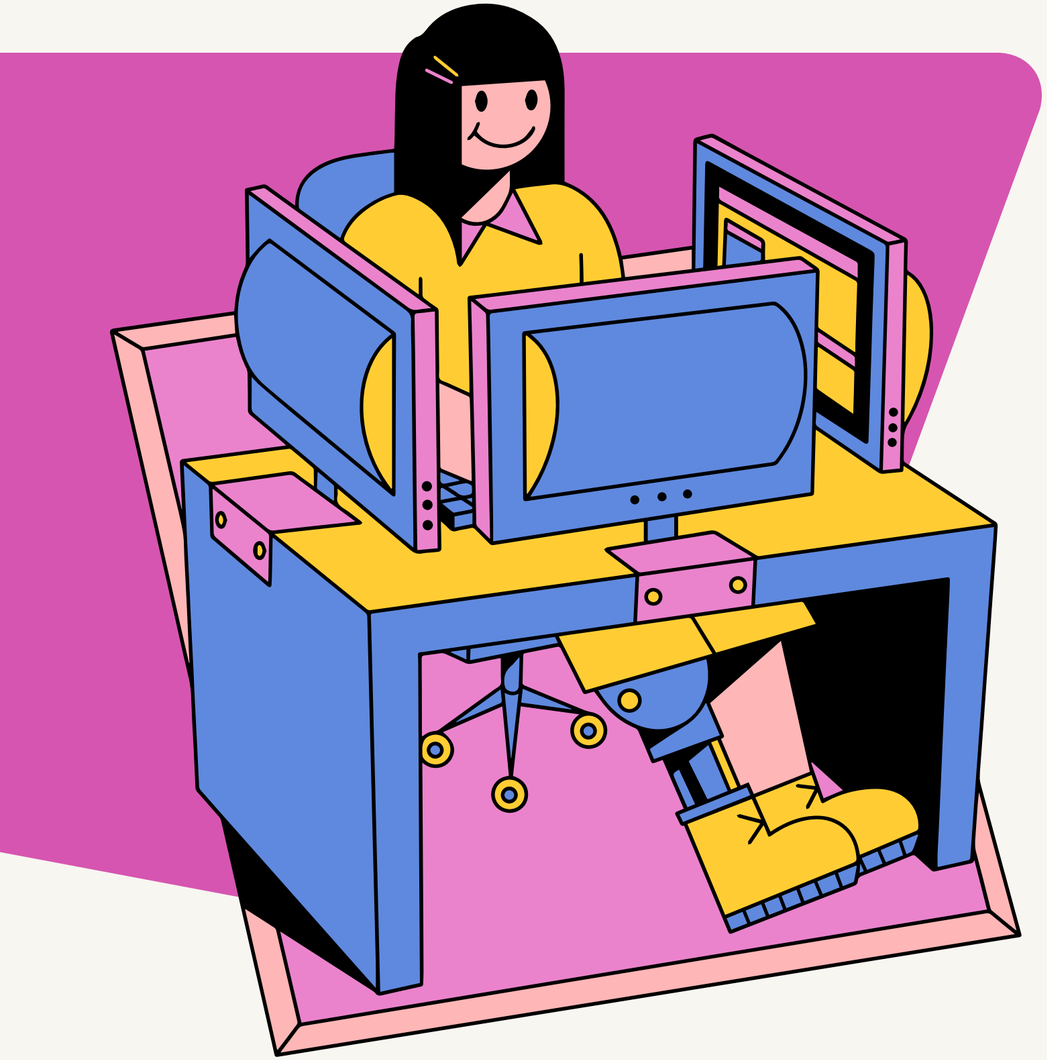
Syntax:

Constructor(): member(value), member(value) {}

MEMBER INITIALIZATION LIST

WHY USE INITIALIZATION LIST

- Initialize const data members
- Initialize reference variables
- Initialize objects without default constructor
- Initialize base class members
- Improve performance



QUIZ 1 (QUICK CHECK)

Choose the correct option:

1. What does the this pointer store?
   - a) Value of object
   - b) Address of object
   - c) Copy of object
   - d) Reference to class

2. **Can this be used inside static functions?**
   - a) Yes
   - b) No

3. **How many copies of a member function exist?**
   - a) One per object
   - b) One per class

ANSWERS:

1. **B**
2. **B**
3. **B**

PRACTICE SLIDE (INITIALIZER LIST)

1. Why can't const data members be initialized inside constructor body?
2. Modify a constructor to initialize:
   - const int id
   - string name
3. Write a constructor using initializer list when parameter names match data members.

```
#include <iostream>
using namespace std;

class Student {
private:
  const int id;
  string name;

public:
  Student(int i, string n) : id(i), name(n) {}
  void print() const {
  cout << "ID: " << id << ", Name: " << name << endl;
  }
};

int main() {
 Student s1(101, "Ali");
 s1.print();
 return 0;
}
```

QUIZ 2 (CONST CONCEPTS)

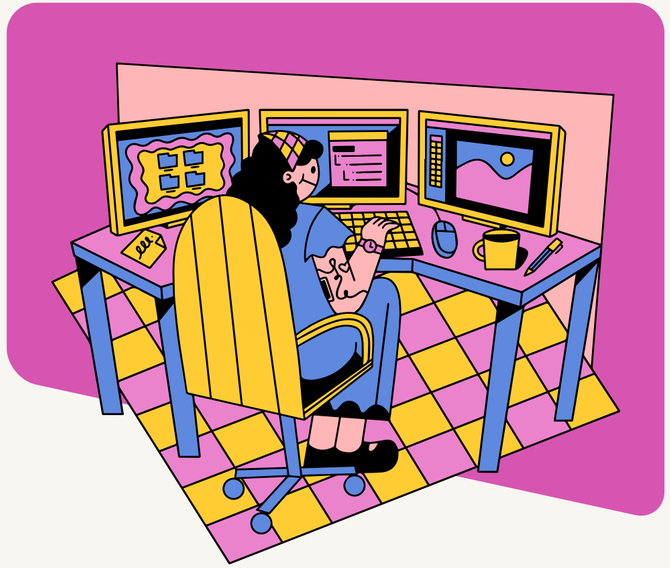
1. **Which function can a const object call?**
   - a) Any function
   - b) Only static functions
   - c) Only const functions

2. **Can constructors be const?**
   - a) Yes
   - b) No

3. **Where is const written in a const member function?**
   - a) Before function name
   - b) After parameter list

ANSWER

1. **C**
2. **B**
3. **B**



STATIC DATA MEMBERS AND STATIC MEMBER FUNCTIONS

Definition:(STATIC DATA MEMBER)

- A single shared variable for all objects of a class

Key Points:

- Declared inside class
- Defined outside class
- Memory allocated once

Definition:(STATIC MEMBER FUNCTION)

- Functions that belong to class, not object

Rules:

- Can access only static data members
- Called using:

ClassName::function();

QUIZ 3 (STATIC CONCEPTS)

- **How many copies of a static variable exist?**
- a) One per object
- b) One per class

- **Static functions can access:**
- a) All members
- b) Only static members

- **Static members are accessed using:**
- a) Object name
- b) Class name and ::

- **ANSWERS**

1. **B**
2. **B**
3. **B**

PRACTICE QUESTIONS

1. Create a class where:
   - ID is constant
   - Name can be changed
2. Track number of objects created using a static variable.
3. Write a class with a const member function that only displays data.

```
#include <iostream>
using namespace std;

class Employee {
private:
  const int id;      // constant ID
  string name;       // name can be changed
  static int count;  // static variable to track objects

public:
  // Constructor using initializer list
  Employee(int i, string n) : id(i), name(n) {
    count++;       // increment object count
  }

  // Setter to change name
  void setName(string n) {
    name = n;
  }
}

// Const member function to display data
void display() const {
cout << "ID: " << id << ", Name: " << name << endl;
}

// Static function to get object count
static int getCount() {
return count;
}
};

// Definition of static member
int Employee::count = 0;

int main() {
 Employee e1(101, "Ahmed");
 Employee e2(102, "Ali");

 e1.setName("Ahsan");

 e1.display();
 e2.display();

 cout << "Total Objects Created: " << Employee::getCount() << endl;
```

PRACTICE QUESTION

Create a class LibraryMember with the following requirements:

Data Members

- const int memberId
  - Must be initialized once and never changed
- string memberName
  - Can be updated
- int booksIssued
  - Initially set to 0
- static int totalMembers
  - Keeps track of how many members are created
- static const int maxBooks
  - Maximum books a member can issue (value = 5)

Functional Requirements

1. Use member initializer list to initialize all required data.
2. Increment totalMembers every time a new object is created.
3. Create a function issueBook() that:
   - Increases booksIssued by 1
   - Does not allow issuing more than maxBook
- Create a const member function display() that prints:
  - Member ID
  - Member Name
  - Books Issued
- Create a static member function getTotalMembers() that returns total members.
- Ensure the program follows const correctness completely.

Main Function Tasks

- Create at least two LibraryMember objects
- Issue books to one member
- Display details using display()
- Print total members using static function

THANK YOU